

COGS 209: Scientific Data Analysis and Statistical Learning

Mini Project 1:
Detecting Scams in Job Postings

Lead Author: Isabella Mullen

Co-Authors: Luna Bellitto, Nina Rice, Asad Tariq

Group Name: Our Group

University of California, San Diego

June 12, 2025

Contents

1	Introduction	2
2	Methods	2
2.1	Data	2
2.2	Cleaning	2
2.3	Exploratory Data Analysis	3
2.4	Preprocessing	4
2.5	Classification Models	5
3	Results	6
3.1	Model Comparison	6
3.1.1	Support Vector Classifier (SVC) Model	7
3.1.2	Logistic Regression Model	7
3.1.3	XGBoost Model	8
3.1.4	Random Forest Model	8
3.2	Visualization	9
4	Discussion	11
5	Bibliography	12
6	Appendix	13

1 Introduction

The proliferation of fake job postings has become a growing global concern. The widespread use of online platforms for recruitment has created new opportunities not only for legitimate hiring but also for fraudulent activity. As the use of these platforms has grown, agencies have started using Applicant Tracking Systems (ATS), software used to streamline hiring, advertise jobs, and manage candidates (Vidros et al., 2017). However, these systems are increasingly being exploited by scammers to collect personal information from job seekers and for financial gain, often damaging the reputations of genuine recruiters in the process (Vidros et al., 2017). Fake job postings have also risen in number as more hiring managers publish jobs to gather resumes or foster a feeling of replaceability in current employees. According to a Resume Builder survey of 650 recruiters, 45% of them admitted to engaging in this practice, with 45% of them posting up to five fake jobs, and 13% posting as many as 75 (Cerullo, 2024). These deceptive practices by both scammers and recruiters significantly hinder the ability of job seekers to identify legitimate employment opportunities.

Our project aims to build a statistical model capable of discerning between legitimate and fraudulent job postings. To do so, we apply several classification models, including Logistic Regression (LR), Support Vector Classifiers (SVC), Random Forest (RF), and Gradient Boosting (XGB), which are used to analyze the job postings dataset. Our primary hypothesis is that the Gradient Boost model will outperform the others as it is particularly effective in handling imbalanced datasets like ours. Moreover, the ability to capture complex patterns and reduce bias through iterative learning makes the gradient boost model particularly well-suited for the detection task. Features of the job postings, such as *has_company_logo*, *required_education*, *telecommuting*, and *description*, are included in the classification models to help infer whether a job is real or fake. We expect that a job posting from a company without a logo would be less reliable, and hence, could indicate a fake job. Similarly, a lack of stated education requirements for a job could also signal a fraudulent job posting.

2 Methods

2.1 Data

This project used a dataset of job listings presented in (Vidros et al., 2017) and obtained through Kaggle (Bansal, 2019). This dataset contains 18,000 job descriptions, with around 800 classified as fraudulent. The data consists of textual details and meta-information about each job posting. We extract the features from this dataset that are relevant to our research question, such as the location of the job, the type of job, the department, the expected education level for the job, and the required experience for the job. We test several solutions to approach the imbalance between the categories. See the Appendix on page 13 to see all columns present in the data.

2.2 Cleaning

The cleaning began by loading and cleaning the raw dataset. We remove job posts with missing data in the description column, as this will be our primary feature variable. We also dropped irrelevant columns such as *job_id*. Then, all text fields were consolidated into a new textual feature column for analysis.

2.3 Exploratory Data Analysis

The main issue of this dataset is class imbalance. This imbalance can lead to biased models that favor the majority class, resulting in high overall accuracy but poor performance in identifying the minority class. Addressing class imbalance is crucial for improving recall, precision, and overall model fairness. We also investigate the extent of missing data. The columns *salary_range*, *department*, and *required_education* have a large percentage of missing data. We created binary indicators for missingness and reshape the DataFrame into a long format suitable for grouped analysis. This horizontal bar plot (Figure 1), enabled direct comparison of missing data patterns across the two classes. Furthermore, Figure 2 highlights the aforementioned class imbalance of our dataset, with only 865 fraudulent jobs as compared to 17,014 real ones. This led to only 4.83% of the job postings being in the minority class.

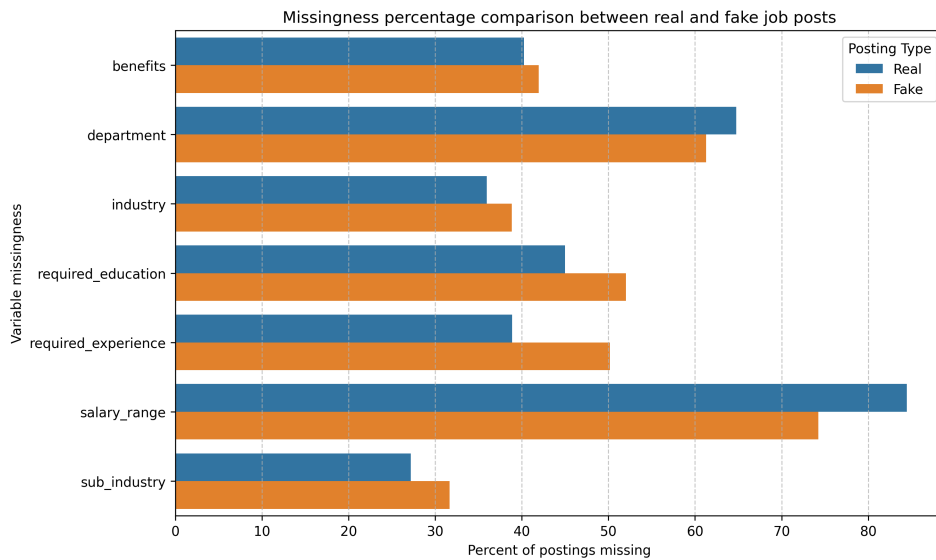


Figure 1: Missingness percentage comparison between real and fake jobs.

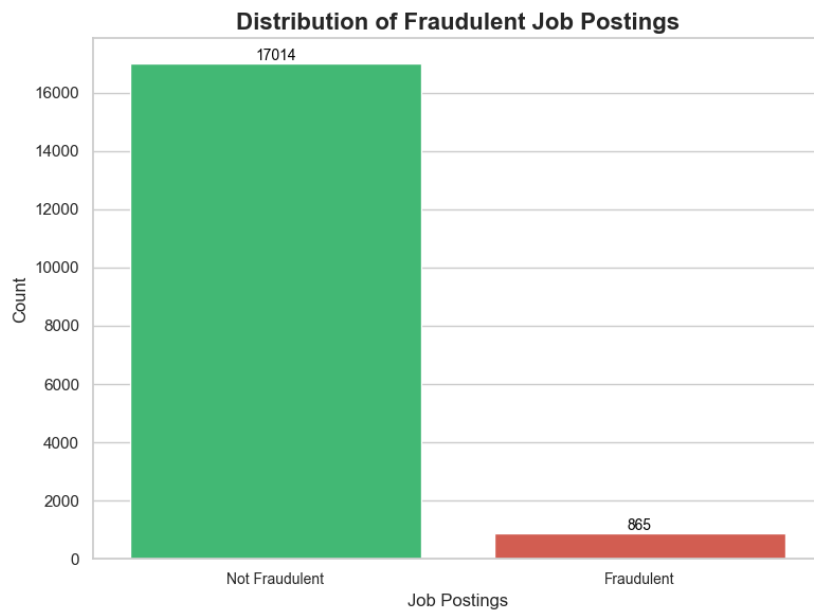


Figure 2: Distribution of fraudulent and non-fraudulent job postings in the dataset.

To explore textual patterns in the dataset, we generate word clouds for both the fraudulent and real job descriptions. The most frequent terms across both classes were largely similar, including general words such as “position”, “experience”, and “looking.” While there is substantial overlap in vocabulary, the visualizations provide a high-level overview of commonly used language. This may reflect subtle differences in tone or emphasis that are explored more systematically in later modeling.



Figure 3: Word cloud of fraudulent job descriptions. The most frequent terms reflect general job advertising language. While these are not unique to scams, their presentation may differ in tone or context.



Figure 4: Word cloud of real job descriptions. Similar vocabulary is found across both classes, though real postings may embed these terms in more structured or detailed descriptions. The overlap illustrates the limitations of surface-level text analysis.

2.4 Preprocessing

To systematically transform the data for the classification models, we take three approaches: one for text, one for numeric features, and one for categorical features. The text pipeline applied a *TfidfVectorizer* with a vocabulary limited to 2,000 features. This process excluded English stopwords and used a token pattern to capture whole words. Tokenizers are tools that split text into smaller units for us in textual analysis and vectorization. The numeric pipeline used *SimpleImputer* for missing data, which fills in any *NAs* with the most commonly occurring value, followed by *StandardScaler* to standardize

the scale of numeric inputs. The categorical pipeline imputes missing values using the most frequent category, then applied *OneHotEncoder*, which converts each categorical variable into a set of binary (0 or 1) columns—one for each unique category level. This approach prevented the model from assuming any ordinal relationship between category values by treating each as an independent feature. One-hot encoding is especially effective for algorithms that do not natively handle categorical variables, such as logistic regression or support vector machines. In this pipeline, the *handle_unknown='ignore'* parameter ensured that if new, unseen categories appear during prediction, they will be safely ignored rather than causing an error.

These individual pipelines are unified using a *ColumnTransformer*, which applied the text transformer to the *combined_text* column, the numeric transformer to a defined list of numeric features, and the categorical transformer to categorical features. This modular design ensured that all feature kinds are cleaned, imputed, and transformed in a unified and replicable way.

2.5 Classification Models

To test our hypotheses, we used classification models present in Python's *sklearn* library. For each classification model under consideration, a systematic search is performed over a predefined grid of hyperparameters. This was done using stratified 5-fold cross-validation to ensure that class distributions are preserved across training and validation folds. This is especially important for imbalanced datasets as in our case. During the tuning process, both F1 score and accuracy were computed for each parameter combination; however, the F1 score is used as the primary criterion for selecting the best model configuration, as it balances precision and recall. After fitting the grid search for each model on training data, the model configuration with the best F1 score for each type of model is stored after retraining.

The classification models considered are as follows:

Support Vector Classifier (SVC)

SVC is a powerful algorithm that constructs a decision boundary maximizing the margin between classes. It is particularly effective at modeling complex, non-linear relationships, making it suitable for nuanced classification problems.

Logistic Regression (LR)

LR is a probabilistic model that estimates the likelihood of class membership using a logistic function. Its interpretability, low variance, and efficiency make it a reliable baseline model for binary classification tasks.

XGBoost (XGB)

XGB is a gradient boosting algorithm that builds a series of decision trees, optimizing for prediction accuracy and using regularization to prevent overfitting.

Random Forest (RF)

RF is a method that constructs multiple decision trees and aggregates their predictions through majority voting. It is resilient to overfitting, handles feature interactions well, and performs reliably across a wide range of classification problems.

In order to mitigate the effects of class imbalance, we used specific balancing techniques directly into our models. For Logistic Regression, Random Forest, and Support Vector Classifier, we set the *class_weight='balanced'* parameter, which automatically adjusts weights inversely proportional to class frequencies in the data. This ensured that the minority class is given greater importance during training. For XGBoost, we manually calculate the *scale_pos_weight* parameter based on the ratio of majority to minority class samples. These adjustments helped the models better detect fraudulent postings.

All four models are trained on the training subset of the job postings dataset, and the parameters for each model that resulted in the best F1 score were saved and used on the testing subset.

The hyperparameters considered for each model are as follows:

- SVC: $C = [0.01, 0.1, 1, 10]$, $decision_function_shape = [ovr]$, $gamma = [scale]$, $kernel = [rbf]$
- LR: $C = [0.01, 0.1, 1, 10]$, $penalty = [l2]$, $class_weights = [balanced]$
- XGB: $learning_rate = [0.01, 0.1]$, $max_depth = [3, 6]$, $n_estimators = [100, 200]$
- RF: $max_depth = [None, 10, 20]$, $min_samples_split = [2, 5]$, $n_estimators = [100, 200]$

3 Results

3.1 Model Comparison

Figure 5 compares the training F1 scores achieved by the four classification models used to detect fraudulent job postings. Among these, the Support Vector Classifier (SVC) demonstrated the highest performance with an F1 score of 0.86. Logistic Regression and XGBoost performed comparably, each achieving F1 scores in the range of 0.78 to 0.79. The Random Forest classifier yielded the lowest F1 score at 0.75, indicating relatively weaker performance on this classification task.

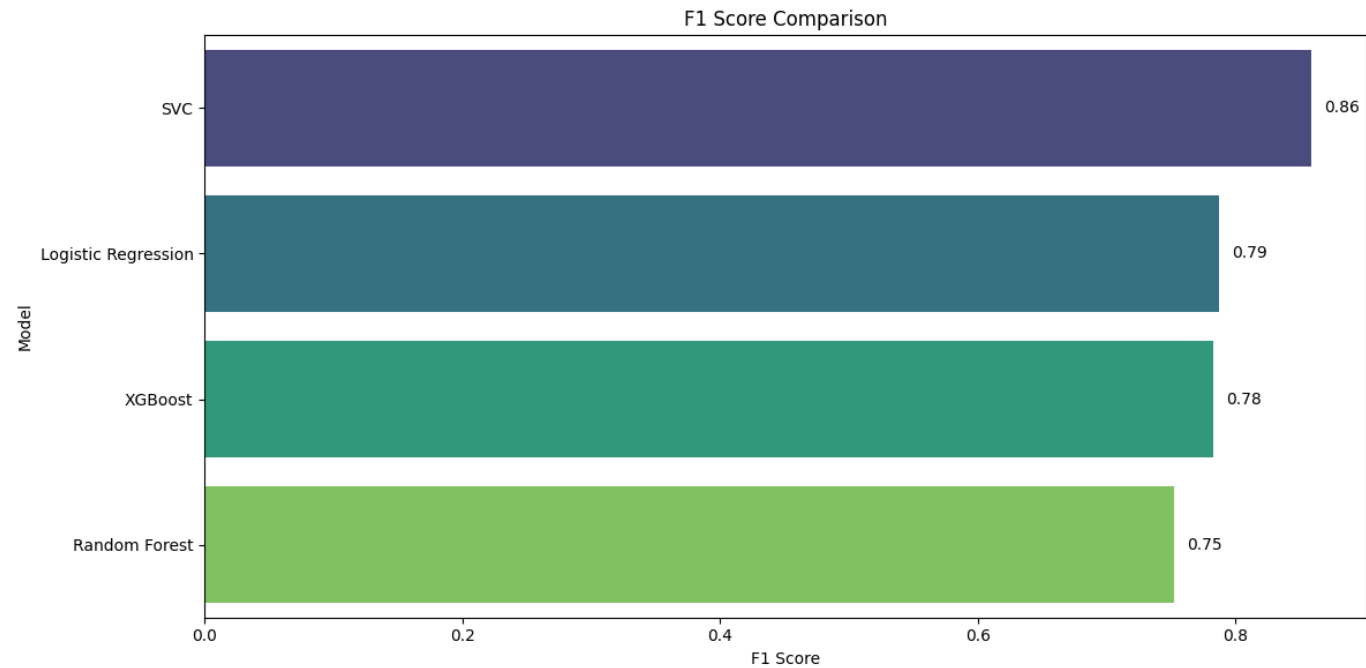


Figure 5: F1 Scores of the SVC, Logistic Regression, XGBoost and Random Forest Classification Models. Parameter combinations returned by Grid Search CV.

3.1.1 Support Vector Classifier (SVC) Model

Name	C	Decision Function	Gamma	Kernel	Mean F1	Mean Accuracy
SVC	10	ovr	scale	rbf	0.810	0.983
SVC	1	ovr	scale	rbf	0.708	0.967
SVC	0.1	ovr	scale	rbf	0.362	0.850
SVC	0.01	ovr	scale	rbf	0.246	0.787

Table 1: Performance results for the Support Vector Classifier, sorted based on mean training F1 Scores for different parameter values.

Out of the four SVC classification models, the one with the highest mean training F1 score has a C value of 10. The model gives more importance to minimizing classification errors, allowing fewer margin violations. This resulted in better and tighter decision boundaries for classification.

This SVC model used the radial basis function (*rbf*) kernel that is good at capturing non-linear relationships of the features in the model. The decision function, *ovr*, maintained the default for binary classification using SVCs.

3.1.2 Logistic Regression Model

Name	C	Penalty	Class Weights	Mean F1	Mean Accuracy
Logistic Regression	10	l_2	balanced	0.713	0.968
Logistic Regression	1	l_2	balanced	0.624	0.949
Logistic Regression	0.1	l_2	balanced	0.428	0.885
Logistic Regression	0.01	l_2	balanced	0.290	0.810

Table 2: Performance results for the Logistic Regression Classifier, sorted based on mean training F1 Scores for different parameter values.

The Logistic Regression model with the highest mean training F1 (0.713) has a high C value of 10. This indicates that the model applied weaker regularization, allowing more flexibility in fitting the training data. As a result, the model is better able to capture important patterns in the data, which improves its classification performance.

The best Logistic Regression model also used the l_2 penalty, which ensured that the coefficients are not overly large while also ensuring that sparsity is not forced. Due to the dataset being inherently imbalanced, balanced class weights were used in the model, which helped to adjust the class imbalance by assigning higher weights to the minority class, which in our case was the fraudulent job postings (*fraudulent* = 1).

3.1.3 XGBoost Model

Name	Learning Rate	Max Depth	Estimators	Mean F1	Mean Accuracy
XGBoost	0.10	6	200	0.738	0.972
XGBoost	0.10	6	100	0.664	0.959
XGBoost	0.10	3	200	0.608	0.946
XGBoost	0.10	3	100	0.527	0.923
XGBoost	0.01	6	200	0.499	0.914
XGBoost	0.01	6	100	0.455	0.897
XGBoost	0.01	3	200	0.404	0.880
XGBoost	0.01	3	100	0.374	0.866

Table 3: Performance results for the XGBoost Classifier, sorted based on mean training F1 Scores for different combinations of learning rate, max depth, and number of estimators.

Out of the eight XGBoost classification model variants, the one with a learning rate of 0.10, maximum depth of 6, and 200 estimators has the highest mean training F1 score (0.738).

The learning rate of 0.10 ensured speedy learning on the training subset and avoids the risk of overfitting. A maximum depth of 6 allows for the trees in the XGBoost model to learn feature interactions better. To improve overall classification performance, we kept the number of estimators at 200 for the model so that the predictions could be improved iteratively.

3.1.4 Random Forest Model

Name	Max Depth	Min Samples Split	Estimators	Mean F1	Mean Accuracy
Random Forest	–	5	100	0.686	0.976
Random Forest	–	5	200	0.677	0.975
Random Forest	–	2	200	0.676	0.976
Random Forest	–	2	100	0.675	0.976
Random Forest	20	5	100	0.629	0.958
Random Forest	20	2	100	0.626	0.958
Random Forest	20	5	200	0.625	0.957
Random Forest	20	2	200	0.622	0.957
Random Forest	10	5	200	0.549	0.934
Random Forest	10	2	200	0.545	0.934
Random Forest	10	5	100	0.541	0.933
Random Forest	10	2	100	0.540	0.932

Table 4: Performance results for the Random Forest Classifier, sorted based on mean training F1 Scores for different parameter values. “–” indicates ‘None’ for max depth.

The Random Forest model achieved the highest mean training F1 score (0.686). The random forest had a minimum sample split of 5, 100 estimators, and no predefined value for maximum depth.

By not limiting the depth of the trees in the model, it captures the complex interactions between the different model features. The trees in the model are also required to have at least five samples for each of their nodes before splitting, which reduces overfitting.

3.2 Visualization

Model	F1 Score	Accuracy	AUC	Precision	Recall
Logistic Regression	0.788	0.975	0.992	0.671	0.954
Random Forest	0.753	0.980	0.993	0.947	0.624
XGBoost	0.783	0.977	0.988	0.730	0.844
SVC	0.859	0.987	0.986	0.915	0.809

Table 5: Final performance metrics on the test set for the best version of each model.

Table 5 shows the key performance metrics obtained from fitting the four models on the testing subset of the job postings dataset, and Figure 6 provides the corresponding confusion matrices of each of the models' classification results.

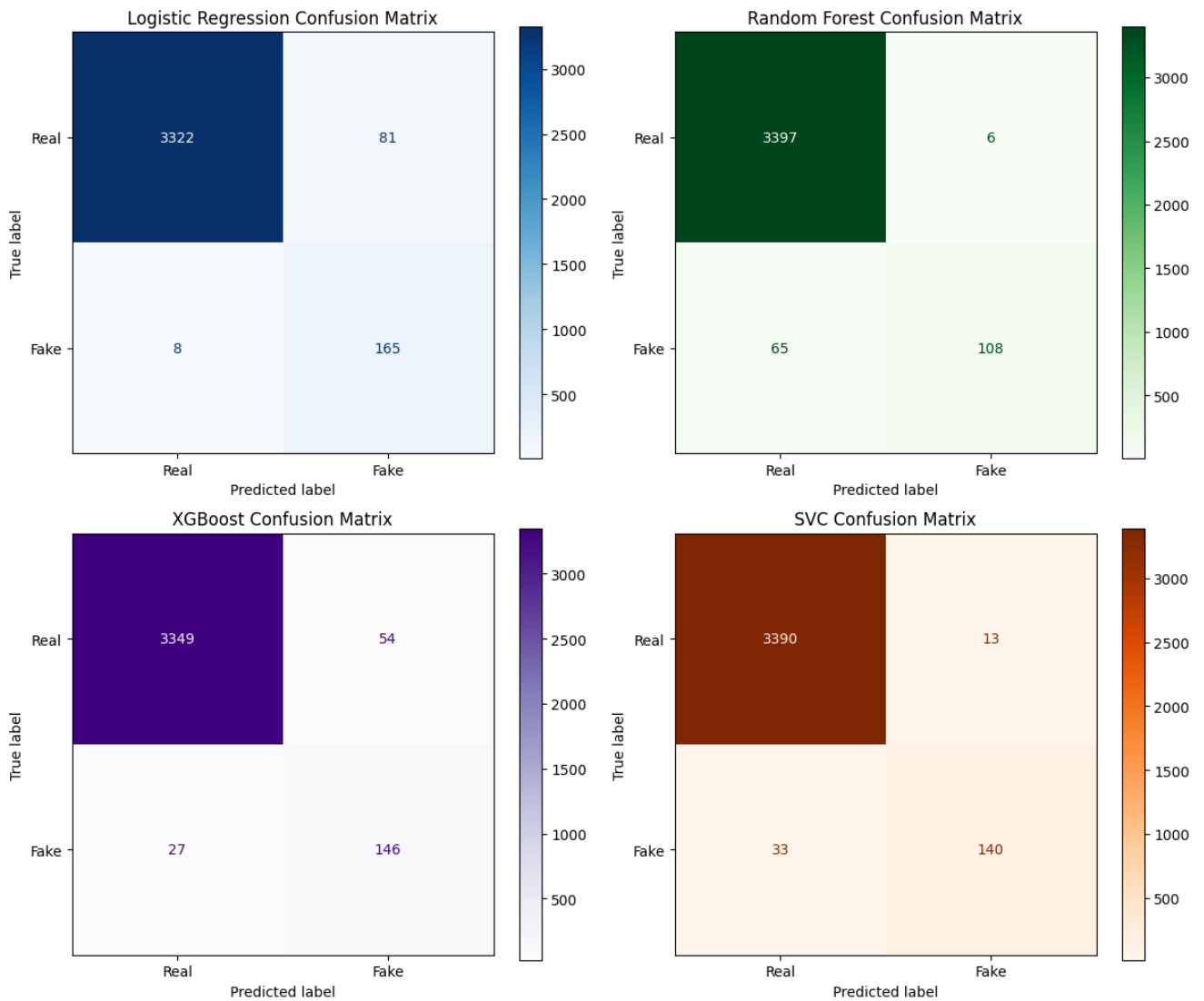


Figure 6: Confusion Matrices of the SVC, Logistic Regression, XGBoost and Random Forest Classification Models.

The best overall ratio between precision and recall was achieved by the SVC model, having a precision of 0.915 and a recall of 0.809, which is also shown in its confusion matrix. Only 13 real jobs were

incorrectly classified as fake (false positives) and 33 fake ones as real (false negatives). This indicated that the model was good at detecting fraudulent jobs, as shown by the fact that it had the highest F1 score (0.859) and the highest accuracy (0.987).

Although the Logistic Regression model had a low precision value (0.671), it had the highest recall (0.954). This model fails to correctly classify only 8 fake job postings. However, the trade-off with this model is a high number of false positives, as it classifies 81 actual real jobs as fake. This is why the precision of this model is the lowest out of the four models.

The XGBoost model performs well overall with a test F1 score of 0.783 and balanced precision-recall scores of 0.730 and 0.844, respectively, which are between those of the SVC and Logistic Regression models. The corresponding confusion matrix shows the robustness of the model with fewer false positives (54) than that of the Logistic Regression model and fewer false negatives (27) than the SVC model.

The Random Forest is the weakest performing model in terms of the test F1 score (0.753). This is because it has a high number of false negatives (65), as seen from the confusion matrix. However, it has the best precision score with only 6 misclassifications of real job postings. This showed that the Random Forest model was careful with classifying jobs as fake, but this results in overlooking many fraud cases.

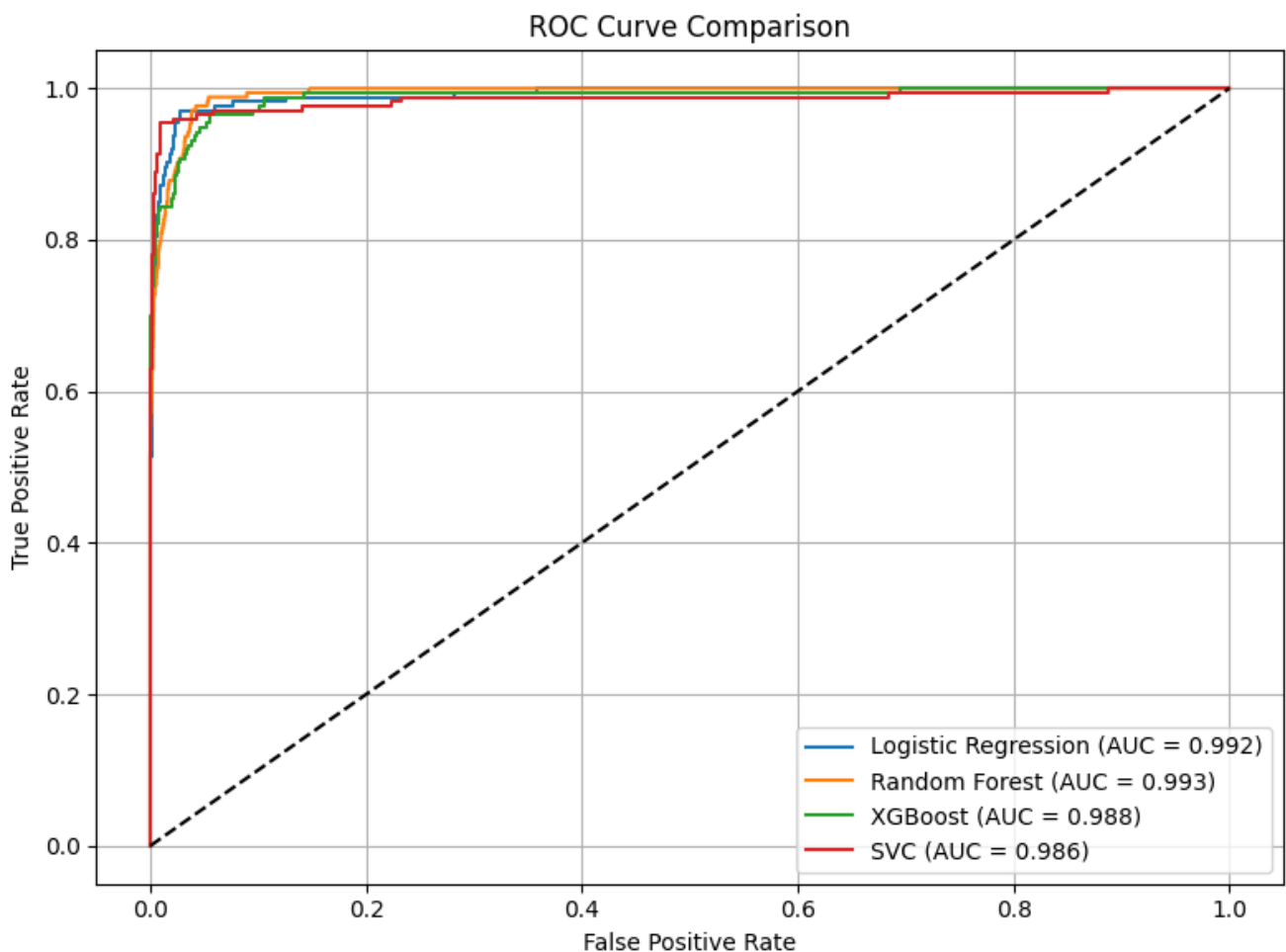


Figure 7: ROC curves of the SVC, Logistic Regression, XGBoost and Random Forest Classification Models with AUC values.

All four models exhibited strong ROC performance with AUC values above 0.98, as shown in Figure 7, indicating a high sensitivity and a low false positive trade-off and reflecting good separability.

4 Discussion

This study set out to develop a statistical model capable of accurately distinguishing between legitimate and fraudulent job postings by analyzing a dataset containing both structured metadata and unstructured textual data. Four classification models, Logistic Regression, Random Forest, Support Vector Classifier, and XGBoost, were evaluated with the goal of identifying the most effective approach for detecting fake postings. To optimize performance, each model underwent hyperparameter tuning through grid search and 5-fold cross-validation. Model evaluation was based on multiple metrics, including F1 score, accuracy, precision, recall, AUC, and confusion matrices.

We utilized a dataset comprising 17,897 job postings, of which 865 belonged to the fraudulent category. Given the class imbalance in the dataset, the F1 score was prioritized as the most informative performance metric because it offers a balance between precision and recall. Using model accuracy alone would not be reliable. The SVC emerged as the top-performing model, achieving an F1 score of 0.86 and an accuracy of 0.987 on the test set. These results were further validated by the confusion matrix, which indicated strong classification performance and a low rate of misclassification.

The Random Forest classifier attained an AUC of 0.993, surpassing the rest of the models in this respect. However, its lower F1 score (0.75) and higher false positive rate, as revealed in the confusion matrix, indicate its weakness in detecting fraudulent postings. Although Logistic Regression and Gradient Boosting achieved F1 scores of 0.79 and 0.78, respectively, their overall performance was inferior to that of the SVC. The successful performance of SVC, used with the *rbf* kernel, shows its effectiveness in handling complex, nonlinear relationships in the data.

We gained three key insights from our classification task. First, model selection and performance evaluation must account for the highly imbalanced dataset, as accuracy alone can be misleading in such contexts, and other metrics such as the F1 score, precision, and recall might be more informative. Second, tuning hyperparameters and selecting appropriate kernels or model configurations significantly impacts results. Finally, the study affirms that classical machine learning models, when properly optimized, can be highly effective in real-world classification tasks.

Future improvements to our work could include expanding the dataset with job postings from various other platforms, thereby enhancing the generalizability of our classification models, considering differences in the text used for job descriptions and other metadata. Furthermore, models, such as Naive Bayes, could be utilized to improve performance.

5 Bibliography

Shivam Bansal. Real or fake job posting prediction. <https://www.kaggle.com/datasets/shivamb/real-or-fake-fake-jobposting-prediction/data>, 2019. Accessed: 2025-05-10.

Megan Cerullo. That job you applied for might not exist. here's what's behind a boom in "ghost jobs.", June 2024. URL <https://www.cbsnews.com/news/fake-job-listing-ghost-jobs-cbs-news-explains/>. Accessed: 2025-05-11.

Sokratis Vidros, Constantinos Kolias, Georgios Kambourakis, and Leman Akoglu. Automatic Detection of Online Recruitment Frauds: Characteristics, Methods, and a Public Dataset. *Future Internet*, 9(1):6, March 2017. ISSN 1999-5903. doi: 10.3390/fi9010006. URL <https://www.mdpi.com/1999-5903/9/1/6>.

6 Appendix

Columns Present in the Job Postings Dataset

- job_id: Unique Job ID
- title: The title of the job ad entry.
- location: Geographical location of the job ad.
- department: Corporate department (e.g. sales).
- salary_range: Indicative salary range (e.g. \$50,000-\$60,000)
- company_profile: A brief company description.
- description: The details description of the job ad.
- requirements: Enlisted requirements for the job opening.
- benefits: Enlisted offered benefits by the employer.
- telecommuting: True for telecommuting positions.
- has_company_logo: True if company logo is present.
- has_questions: True if screening questions are present.
- employment_type: Full-time, Part-time, Contract, etc.
- required_experience: Executive, Entry level, Intern, etc.
- required_education: Doctorate, Master's Degree, Bachelor, etc.
- sub_industry: Automotive, IT, Health care, Real estate, etc.
- industry: Consulting, Engineering, Research, Sales etc.
- fraudulent: target - Classification attribute.